

HelixConstitution

HelixConstitution

すべてのプロジェクトが継承する普遍的エンジニアリング憲法——機械的に強制されるアンチブラフ法、**Git**サブモジュールとして共有

概要

HelixConstitutionは、単一のプロジェクト非依存型ルールブックであり、Helix/vasic-digitalプロジェクトのすべてがGitサブモジュールとして追加するものである。これは譲れないエンジニアリング規律（アンチブラフ、エビデンスのみの検証、データ／ホストの安全性、ドキュメントとテストカバレッジ）を成文化し、140以上のリポジトリ群に伝播させる。全体を一貫させるガバナンスの基盤である。

短い説明

Gitサブモジュールとして提供される普遍的かつ継承可能なConstitution。すべての利用プロジェクトが自動的に継承し、拡張は可能だが弱体化は許されない、強制的かつ譲れないルール群——アンチブラフのエビデンスゲート、偽陽性耐性、データおよびホストの安全性、カバレッジとドキュメント規律——を定義する。

長い説明

HelixConstitutionは、Gitサブモジュールとして追加することで参加を選択したすべてのプロジェクトに共有されるエンジニアリングプラクティスの正統かつ唯一の情報源であり、コードと同様に分散されバージョン固定された「エンジニアリング法」である。その核となるConstitution.mdは、約1MBの継続的にバージョン管理された文書で、番号付き条項群

（§11.4.xの契約ファミリー、現在は§11.4.170まで）と、エージェントごとの運用マニュアル（CLAUDE.md、AGENTS.md、QWEN.md、GEMINI.md）から構成される。これらのマニュアルは本文書を参照することで、人間とすべてのCLIEージェントが同一のルールブックを共有する。継承構造は意図的に3層に分かれている：普遍的なベース（このサブモジュール）、プロジェクト層（プロジェクト独自のConstitution/CLAUDE/AGENTSで拡張可

能)、そしてオプションのサブディレクトリ層である。これらは上から下へ評価され、プロジェクトはルールを強化することはできるが、弱体化することは構造的に禁じられている。その結果、140以上のリポジトリ群は、共有する規律が固定されているため、静かに分裂することはあり得ない。

この文書は徹底的にドメイン非依存である。特定のベンダー名、ハードウェアSKU、ポート番号、ライブラリバージョンを記載するものはすべて、利用プロジェクト独自のConstitutionに移行しなければならない、普遍性は決して前提とされない——ルールがベースに組み込まれる前に、明確な4つのテストに合格することで獲得されなければならない。その哲学的な背骨はアンチブラフであり、相互に連携する契約群——§1.1偽陽性耐性、§11.4エンドユーザー品質契約、§11.4.6ノーギャンブル、§11.4.6.9ポジティブエビデンス分類——によって表現される。これらの組み合わせ効果は単一の明確な基準を生み出す:「テストが通る」ではなく、「実際のユーザーが機能を使える」ことがリリースの条件であり、すべての合格結果は物理的なエビデンスを引用しなければならない。付属のsubmodules-catalogue.md (142リポジトリ) は、「すでに同じ機能を持つものを保有しているか?」という問いを、新たなコードを書く前にカタログ優先の「拡張せよ、再実装するな」という反射的行動に変える。ヘルパースクリプトはサブモジュールを任意のネスト深度から検出し、すべてのコミットを4つの独立したGitプロバイダーに同期させるため、唯一の権威あるルールブックを失うことは不可能である。

コンテンツ

なぜこれを構築したのか

同じ所有者が手がける複数の大規模プロダクトアプリと、数十に及ぶ分離された再利用可能なサブモジュールが、同じ苦勞して得たルールを何度も再発見し続け——そして同じ種類の失敗を繰り返し続けていた。それは、テストやステータスレポートが成功を主張しているにもかかわらず、エンドユーザーにとっては機能が壊れているというもの(「PASS偽装」や「FAIL偽装」)。Constitutionの各フォレンジックアンカーは、実際のインシデントを記録している(例えば、2026年5月20日のD3オーディオルーティングにおけるPASS偽装。検証は「コーデック使用中」フィールドが空のままグリーンになり、あるいは2026年6月25日の巨大ボタンUIがトークン等価性テストに合格したにもかかわらず、実際の画面が壊れていたケースなど)。Constitutionは、こうした不誠実な成功を機械的に不可能にするために存在する——一度、普遍的に。これにより、規律がプロジェクト間で揺らいだり、静かに忘れ去られることがない。

なぜゲームチェンジャーなのか

これはエンジニアリング文化を、「従うことを期待されるドキュメント」から「継承され、バージョン管理され、機械的に強制される法」へと変える。スタイルガイドとコンパイラの違いに等しい。一つのサブモジュールのバージョンアップで、全フリーットのルールが一度に、アトミックかつ追跡可能な形で更新される。単一のアンチ偽装契約は、信頼ではなく構造によって、すべての利用リポジトリに保証される。伝播ゲートはフリーット全

体で条項番号を文字通りgrepし、ペアとなるミュートーションテストがゲート自体が偽装でないことを証明する——つまり、強制さえもが強制される。ガバナンスは、誰も読まないウィキ上の理想ではなく、CIジョブに指し示せる監査可能でテスト可能な事実となる。

何が革新的なのか

- **Constitution-as-submodule** — エンジニアリング法を、コードとまったく同じように配布・バージョン固定する。意図的なv1.0.0形式のタグとプロジェクトごとのピン留めにより、すべてのリポジトリは正確にどのバージョンの法に拘束されているかを知る。
- アンチ偽装を第一級のフォレンジック原則として — すべての条項は、逐語的なオペレータの指示、そしてしばしばそれを動機づけた実際のインシデントにまで遡る。そのため、ルールブックは意見ではなく判例のように読める。
- ルール自体のメタテスト (§1.1) — すべてのゲートには、PASS→FAILに反転させるミュートーションがペアになっている。これにより、「ゲートが偽物でない」ことは主張ではなく、毎回の実行で証明される。失敗し得ないゲートは、ゲートがないよりも悪い扱いを受ける。
- 獲得された普遍性 — あるルールが真に普遍的か、それともプロジェクト固有のものかを判断する明確な4部構成のテストにより、ベースをシンプルに保ち、ポータブルでベンダー依存のない状態を維持する。

すべてのプロダクトでどのように使われているか (与えられる力)

必須のガバナンス基盤として、HelixConstitutionはファミリーが参照するドキュメントではない——それはファミリーが構築される基盤そのものだ。

- ガバナンスの背骨：すべてのHelix/vasic-digitalプロジェクトはこれをサブモジュールとして追加し、CLAUDE.md / AGENTS.md / QWEN.md、あるいは独自のConstitution.mdからインポートする。ルールは無条件に適用され、最初のコミットからプロジェクトごとのオプトアウトはない。
- ゲートとマンドート：4層のカバレッジモデル（ソース存在、ビルド耐性、ランタイム挙動、ゲート非偽装）を定義し、機能は4つすべてのレベルをクリアして初めて「完了」とみなされる。さらに、増え続ける名前付きマンドートも存在する：認証情報の取り扱い (§11.4.10)、常時同期ドキュメント (§11.4.60)、コンテナサブモジュール義務 (§11.4.76)、CodeGraph (§11.4.78)、必須テストタイプカバレッジ (§11.4.169) など。
- 伝播：CM-COVENANT-114-NNN-PROPAGATIONゲートは、利用フリース全体に文字通りの条項テキストが存在することを保証する。これにより、契約がエステートの片隅で静かに削除されることはない。非準拠は、回避フラグのないハードなリリースブロッカーとなる。

- 発見: submodules-catalogue.mdにより、「Xを実現する既存モジュールはあるか?」という問いは、新しいモジュールをスキファールドする前に一目で答えが出る。重複努力は根源で断たれる。
- **AI**エージェントの一貫性: 同じ法が、すべてのCLIエージェント (Claude Code、Codex/Cursor/Aider/OpenCode/Crush/Kimi via AGENTS.md、Qwen Code via QWEN.md) に同一の形で表現される。そのため、どのツールがコードに触れても、同じ一つの契約に従う。

内容

最大の技術的課題とその解決方法

- 任意のネスト深度からサブモジュールを特定する — 3階層深く埋め込まれたルールでも、その所在を知らずに法を見つけ出さなければならない → `find_constitution.sh` は親ディレクトリを遡り、Gitのスーパープロジェクトポイントを再帰的にたどり、`CONSTITUTION_DIR` の上書きと2つのサポート対象レイアウト (`constitution/`、`submodules/constitution/`) に対応。ネストがどれほど深くても解決は確定的。
- 4つの**Git**プロバイダー間で単一リポジトリを権威あるものに保つ — ミラーが同期しなければ意味がない → `install_upstreams.sh` は宣言的な `Upstreams/*.sh` リモートを読み込み、`origin` に複数のプッシュURLを設定。単一の `git push` でGitHub (プライマリ)、GitLab、GitFlic、GitVerseへの原子的なファンアウトを実現し、どのミラーも遅延しない。
- ルールの肥大化／プロジェクト固有の漏出をユニバーサルベースに防ぐ — 「とりあえずここに追加」の誘惑は、移植性を損なう → 獲得された普遍性の4段階テストと§11.4.17のユニバーサル対プロジェクト分類をすべての新規ルールに適用。プロジェクト固有の懸念事項は、本来のプロジェクト層に押し戻す。
- 継承ゲートが実際に機能することを証明する — 失敗が見えないゲートは信頼できない → `meta_test_inheritance.sh` という番兵メタテストが、§11.4のアンカーを意図的に削除し、ゲートがそれを検知することをアサート。これにより、強制メカニズム自体が静かな破綻に対して継続的に再検証される。

テックスタック

- **Git**サブモジュール継承
 - 理由: Gitサブモジュールは、ルールブックを権威あるものにかつ消費者ごとにバージョン固定できる唯一のメカニズム。アップグレードは、コピー＆ペーストの静かな繰り返しではなく、明示的でレビュー可能なバージョンアップで行われる。
 - 方法: 消費側プロジェクトはサブモジュールを追加し、そのエージェントファイルを `@import`。3つのレイヤーは上から下へ評価され、すべての境界で「拡張はするが弱体化させない」という厳格な契約が適用される。

- **find_constitution.sh**
 - 理由: ルールが深くネストされたコードから確実に見つけられなければ無意味であり、パスのハードコーディングはプロジェクトの再編成で即座に破綻する。
 - 方法: 親ディレクトリの遡上と `git rev-parse --show-superproject-working-tree` の再帰処理を組み合わせ、`CONSTITUTION_DIR` の上書きでバックアップ。2つのサポート対象レイアウトを解決。
- **install_upstreams.sh + Upstreams/**
 - 理由: 4プロバイダーの冗長性は、維持に余分な労力がかかるなら意味がない。さもないとミラーは陳腐化する。
 - 方法: 宣言的なリモートごとの `.sh` ファイルを、単一のマルチURL origin に具現化。4つのプッシュを1つに集約。
- **§1.1 ミューテーションメタテスト**
 - 理由: 失敗しないゲートは、存在しないよりも悪い。偽りの安心感を生むからだ。
 - 方法: 各ゲートには `sed` による削除／リネームのミューテーションがペアで設定され、`PASS→FAIL`に変化することを確認した後、元に戻される。これにより、すべてのゲートが毎回機能することを証明。
- **伝播ゲート (CM-COVENANT-114-NNN-PROPAGATION)**
 - 理由: 誓約がフラグシップリポジトリだけでなく、すべての消費者に検証可能に存在してこそ、真に普遍的である。
 - 方法: 消費者全体にわたる条項番号のリテラル`grep`を実行。§1.1のミューテーションとペアになっており、伝播チェック自体が失敗することを証明。
- **submodules-catalogue.md (§11.4.74)**
 - 理由: 重複防止の規律を破る最速の方法は、自分がすでに何を持っているかを知らないことだ。
 - 方法: 142リポジトリを機能別にグループ化した目録。新規スキャンフォルディングの前に、目録チェックをトラッカーに記録。
- **マルチフォーマットエクスポート**
 - 理由: 同じ法は、人間が読む場合も、ツールが解析する場合も、アーカイブが保存する場合も、等しく利用可能でなければならない。
 - 方法: すべての正規ドキュメントは、1つのソースから `.md`／`.html`／`.pdf`／`.docx` として出力。

コンテンツ

ステータスおよび正確性に関する注意事項

- ステータス: 出荷済み。フリート全体（公開正規リポジトリおよびミラーリポジトリ）でサブモジュールとしてアクティブにバージョン管理され、使用中。
- ライセンス: 未定 — ソース資料には明記されておらず、公開前にリポジトリのLICENSEファイルで確認のこと。

- 上流ミラー追加先: GitLab helixdevelopment1/helixconstitution、GitFlic helixdevelopment/helixconstitution、GitVerse helixdevelopment/HelixConstitution。

優先度レベル: Helix-プライマリ — Helixファミリーにおけるすべての構築プロセスの必須ガバナンス基盤。